



LLMOps Concepts

LECTURE

LLMOps vs MLOps



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

In this lecture LLMOps vs MLOps, we explore LLMOps concepts, development and deployment patterns, packaging and governance, cost and performance considerations, human feedback, adapting MLOps for LLMs, and the use of Databricks Asset Bundles.

What About LLMOps?

Comparing and contrasting LLMOps from traditional MLOps by area

Dev Patterns	Packaging	Serving	API Governance	Cost and Performance	Human Feedback
The workflows and patterns that practitioners follow when developing machine learning and generative AI solutions	The composition of entire machine learning and generative AI applications in order to effectively deliver and scale solutions	The deployment of machine learning and generative AI applications for output storage or delivery to the end user.	The access control and management of APIs used as a part of the machine learning or generative AI application.	The constant measurement and evaluation of the cost of developing and maintaining the application and its value and performance in production.	The inclusion of data generated by humans into the evaluation or iteration of machine learning and generative AI applications.

Note: We'll walk through each of these areas and map to the LLMOps reference architecture.



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

To compare MLOps with LLMOps, it's helpful to focus on a few important areas as you begin your LLM development journey. First, there are development patterns—these are the workflows and best practices used when building either traditional machine learning or generative AI solutions. Packaging models is another aspect, and although we've touched on it before, it's worth emphasizing how model packaging differs for ML and LLM systems as part of a bigger picture.

Serving models efficiently is crucial, along with ensuring enough storage for prediction outputs. API governance is increasingly important: as more systems rely on APIs for model access, strong controls are needed to prevent misuse and manage infrastructure and hardware responsibly. Costs and performance take on new dimensions, where thinking in terms of tokens—especially with API usage—is essential. Finally, incorporating human feedback remains vital for improving quality and guiding future iterations.

Development Patterns in LLMOps

Comparing and contrasting LLMOps from traditional MLOps

Dev Patterns

Packaging

Serving

API
Governance

Cost and
Performance

Human
Feedback

- **Incremental** development patterns
 - Developers typically **start with off-the-shelf external models and move toward more custom models** through RAG, fine-tuning, or pre-training
- Inclusion of text templates
 - Many interfaces to GenAI solutions **include text templates** to facilitate LLM behavior within the application – these must be developed, iterated on, and managed as part of an LLM pipeline



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

Development in LLMOps is still iterative and incremental, but unlike ML, you're not usually starting by training a model from scratch. Instead, most workflows begin by consuming off-the-shelf proprietary models offered through APIs—for example, from providers like OpenAI or Anthropic. As you gain experience and see which models address your use case, you may move toward customizing models, fine-tuning, or creating solutions tailored to your specific data and tasks.

A key difference with language models is the importance of prompt engineering. Using text templates or prompt templates becomes a critical new aspect; these help track and structure the different styles and methods of prompting each model. Since every model responds best to unique prompts based on its own training, maintaining prompt templates brings rigor and flexibility, ensuring they're a core component in your LLM pipeline.

Packaging Artifacts in LLMOps

Comparing and contrasting LLMOps from traditional MLOps

Dev Patterns

Packaging

Serving

API
Governance

Cost and
Performance

Human
Feedback

- Package entire applications
 - Rather than deploying a single model, large applications need to be packaged and deployed together
 - This can include calls to other LLMs or GenAI applications as a part of a deployed pipeline
- Environment configurations
 - Additional configurations need to be made to the package environment to manage things like other endpoints and embeddings, including testing external systems are working

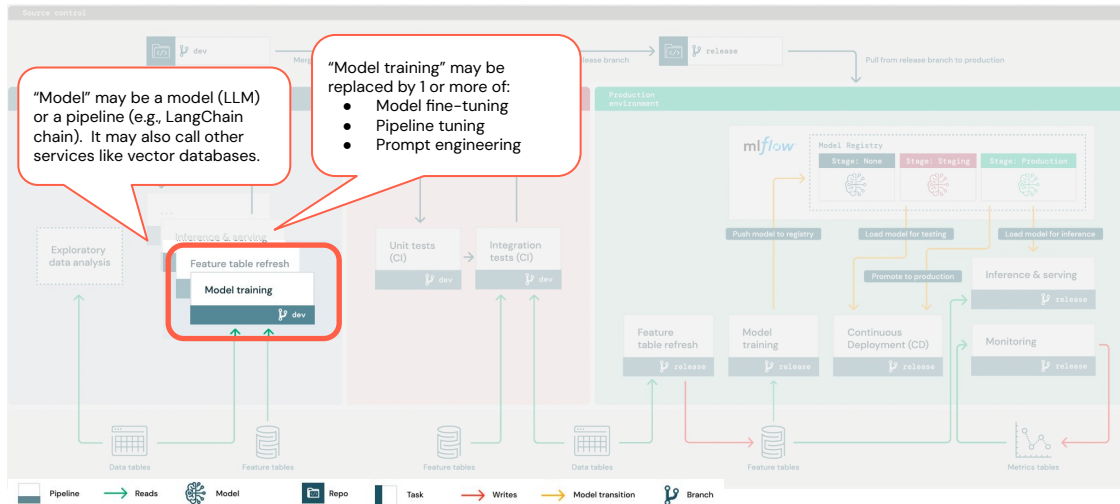


© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

When it comes to packaging, the approach is quite different from traditional ML. Instead of creating a single, atomic artifact that simply takes an input and instantly provides an output, you're often constructing a more complex pipeline or chain. This system may need to interact with various other services and components.

Because of this, each connected service or model in the chain will likely require separate environment configurations, such as unique tokens or access credentials. The packaging process, therefore, needs to account for these dependencies to ensure everything operates smoothly together.

Recommended “deploy-code” architecture (LLMOps)



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

Mapping these topics back to the architecture, dev patterns and packaging mainly sit at the model training stage. Instead of training straight away, you’ll usually begin with prompt engineering. As you progress or have enough data, you may fine-tune models, whether simple or complex—sometimes running locally or interacting with external services. Tying this to the architecture, dev patterns and packaging are part of the model training process. You often start with prompt engineering, then move to fine-tuning as you gain data or experience. Models can range from complex pipelines with external interactions to simple solutions running on local GPUs.

Serving Applications in LLMOps

Comparing and contrasting LLMOps from traditional MLOps

Dev Patterns

Packaging

Serving

API
Governance

Cost and
Performance

Human
Feedback

- Additional components like vector databases
 - When augmenting LLM applications with contextual data, new systems like vector databases need to be set up and used for scale
- GPU infrastructure
 - Due to the size of language models used in many applications, GPUs are required for the serving and deployment of AI systems
- User interface components
 - While some might choose to manage this separately, developers might need to include end user interface components as a part of their serving deployment

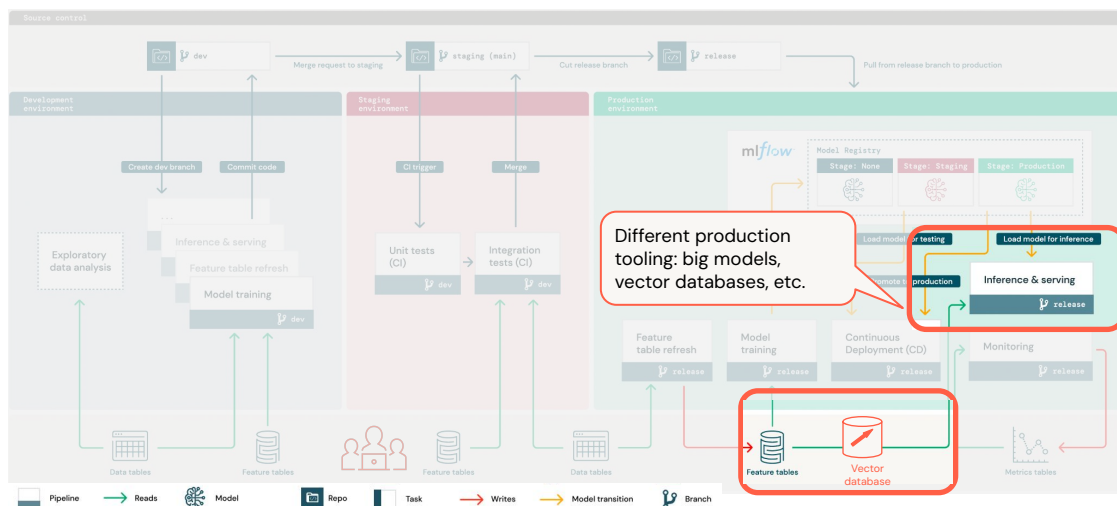


© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

In the serving stack for LLM systems, you're not just deploying a simple CPU endpoint. If you're building a RAG or recommendation system, vector databases become an important new part of your infrastructure, supporting efficient retrieval and augmentation. It's also critical to manage GPU resources if serving your own models, weighing cost and utilization since idle GPUs are expensive and difficult to manage.

Another key point is the need for a user interface—especially when handling multi-turn conversations and collecting human feedback. If you lack a UI/UX team, tools like Gradio or Streamlit make it easier to build applications. Databricks also enables scaling these apps through Lakehouse apps, though deploying containers at scale can still be complex.

Recommended “deploy-code” architecture (LLMOps)



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

In LLM systems, serving goes beyond just standard CPU endpoints. You might be running CPU serving, vector search serving, and even offering user interfaces. This means you’re combining multiple types of production tooling—not just relying on traditional Kubernetes containers, but adding layers to support efficient search and real-time interactions. With LLMs, serving isn’t limited to basic CPU endpoints—you’re often supporting CPU serving, vector search serving, and potentially user interfaces too. As a result, your production environment involves diverse tooling, not just typical Kubernetes containers, to handle these multiple layers.

API Governance in LLMOps

Comparing and contrasting LLMOps from traditional MLOps

Dev Patterns

Packaging

Serving

API
Governance

Cost and
Performance

Human
Feedback

- Managing access to endpoints
 - With the addition of new endpoints being created for various components of the project, access to those endpoints needs to be governed
- Governing use of application
 - Calls to endpoints should be logged and reviewed regularly to evaluate abuse
 - Guardrails can be put in place to prohibit future abuse



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

If you're like most users without the resources or expertise to deploy your own GPUs, you'll rely on APIs for serving models. Managing API access is critical—just like controlling access to data and endpoints—so your tokens and credentials don't get leaked or abused.

Mitigation involves setting rate limits for each token, a feature most API providers offer. In Databricks, when you set up an endpoint that calls an external model, you associate it with a token already configured with specific rate limits, all managed through a unified API system. For users who rely on APIs rather than running their own GPU servers, it's crucial to manage access to those APIs to prevent token leaks and credential overuse. Most providers allow you to create tokens and set rate limits for groups or business units. In Databricks, when spinning up a model endpoint that uses an external service, you assign a rate-limited token—all managed through a single, unified API.

Cost and Performance in LLMOps

Comparing and contrasting LLMOps from traditional MLOps

Dev Patterns

Packaging

Serving

API
Governance

Cost and
Performance

Human
Feedback

- Model size increases cost
 - LLMs are larger in size, and frequently result in more cost
- (API-based) Models have use-based cost
 - (External/FMAPI) Models offers per-token based pricing that needs to be considered
- Techniques for reducing model size/cost
 - Reduce model size
 - Reduce number of queries to external models, if possible (use “caching”)
 - Reduce query input and/or constraint output size

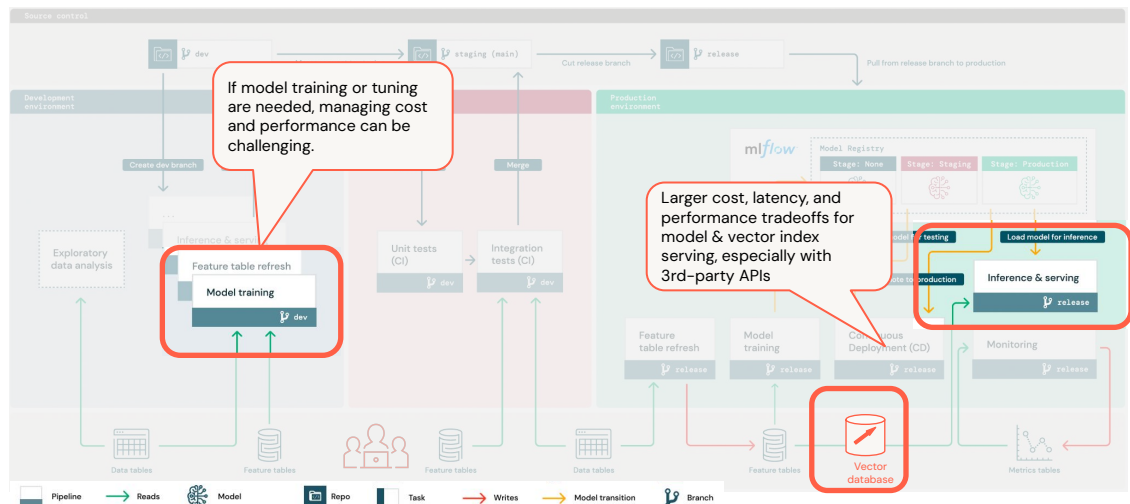


© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

When it comes to cost and performance, larger models demand more infrastructure and result in higher expenses. If you’re using APIs, you enter a “tokenomics” world—paying based on how many input and output tokens you use. Reducing model size and fine-tuning on your specific data can lower costs without sacrificing performance.

Other cost-saving measures include reducing the number of queries by caching repeat questions and trimming input length through summarization or prompt engineering. Fine-tuning generation parameters to set maximum output tokens also helps you control expenses and predict ongoing costs based on historical app usage.

Recommended “deploy-code” architecture (LLMOps)



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

Mapping this back to the architecture, if you’re simply consuming models via prompt engineering or RAG, there are no training costs. Fine-tuning is relatively inexpensive and made easy with tools like Mosaic AI’s fine-tune API. However, pre-training a model can be costly—the larger the model and the more tokens used for training, the higher the expense, especially with factors like number of epochs impacting costs.

Most of the expenses in generative AI systems come from the serving side—whether paying for tokens when using APIs or managing your own infrastructure. This includes keeping endpoints, vector databases, and structured databases like Databricks’ online stores operational and highly available, as these components support prompt augmentation and increase overall system complexity and cost.

Human Feedback in LLMOps

Comparing and contrasting LLMOps from traditional MLOps

Dev Patterns

Packaging

Serving

API
Governance

Cost and
Performance

Human
Feedback

- Collection of data
 - Explicit satisfaction (i.e. with 👍 and 👎)
 - User queries
 - Human/Users corrections/suggestions
- Use of data (managed!)
 - Response examples
 - Tests
 - Architectural/strategic decisions

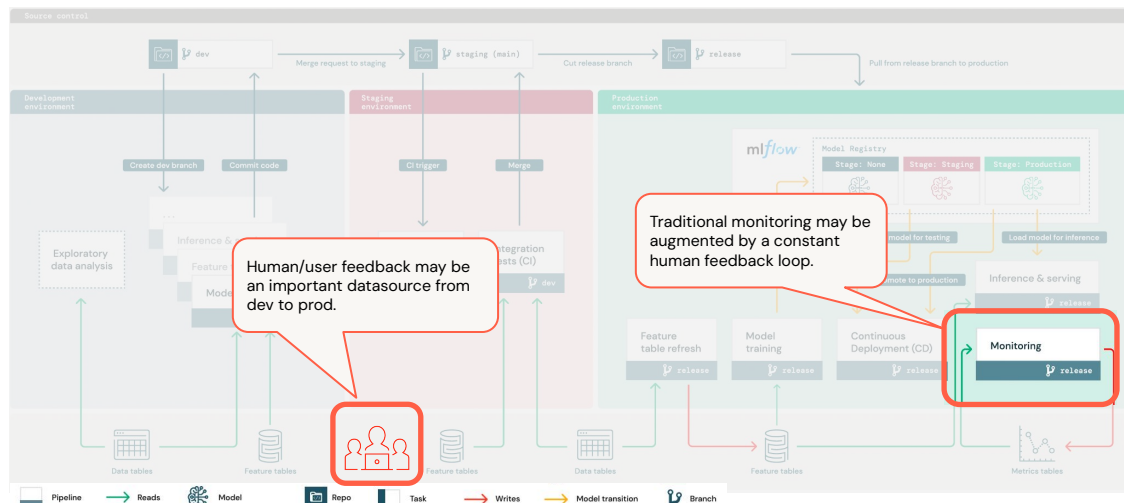


© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

A major new focus in LLMOps is collecting human feedback. Whether in chatbots or other applications, allowing users to rate their experience, provide thumbs up or down, and suggest better answers is extremely valuable. If users contribute this feedback, it offers rich data to improve models.

You can use this human feedback for reinforcement learning or to build higher-quality labeled datasets for fine-tuning, ultimately pushing model quality to the next level.

Adapting MLOps for LLMs



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

Beyond standard monitoring, integrating user feedback into your UI or UX is crucial. By linking feedback directly to data sources, you strengthen the connection between model performance and user experience.

Collecting feedback in production, development, and staging is possible—for example, by inviting power users to test and review your application in staging. This crowdsourced approach helps catch risks and correct issues early, reducing the chance of negative outcomes in the live environment. In addition to traditional monitoring, integrating feedback features into your UI or UX lets users provide valuable input directly tied to specific data. Linking this feedback to your data sources strengthens model evaluation and iteration.

This approach isn't limited to production; it also works in development or staging. Crowdsourcing feedback from power users during testing helps identify and address issues before deployment, making your generative AI systems safer and more reliable.

Databricks Asset Bundles (DABs)



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

We have discussed the LLMOps architecture and the recommended 'deploy-code' approach across development, staging, and production environments. Now, we need the tool that makes this entire process repeatable, reliable, and production-ready. This is where Databricks Asset Bundles, or DABs, come in.

Databricks Asset Bundles

Where are bundles used?

Write code once, deploy everywhere

YAML files that specify the artifacts, resources, and configurations of a Databricks project.

What are Databricks Asset Bundles?

The **new databricks CLI** has functions to **validate**, **deploy** and **run** Databricks Asset Bundles using bundle.yml files

How do bundles work?

Bundles are useful during **development and CI/CD** processes



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

Databricks Asset Bundles provide a solution for the ideal goal of writing code once and deploying it everywhere.

What are Databricks Asset Bundles? They are defined using YAML files that specify all the artifacts, resources, and configurations necessary for a Databricks project.

How do bundles work? The new Databricks CLI includes functions to validate, deploy, and run these bundles using the bundle.yml file. Bundles are particularly useful during development and CI/CD processes as they automate the transition between environments. They consolidate complex setup steps into a concise, declarative syntax.

A Closer Look

Name and default Workspace

Resource configurations

- Jobs, DLT pipelines, MLflow, etc.
- Follows REST API schema

Environment-based specs

- Control project behavior in different environments

```
1 bundle:
2   name: shark_sightings
3
4 workspace:
5   host: https://e2-dogfood.staging.cloud.databricks.com
6
7 resources:
8   jobs:
9     shark_sightings:
10      name: "${bundle.environment} Shark sightings"
11      task_key: shark_sightings
12      tasks:
13        - task_key: shark_sightings
14          notebook_task:
15            base_parameters:
16              dbname: "shark_sightings_${bundle.environment}"
17            notebook_path: ./shark_sightings.py
18            new_cluster:
19              spark_version: 10.4.x-scala2.12
20              num_workers: 1
21              node_type_id: i3.xlarge
22
23 environments:
24   development:
25     default: true
26   production:
27     workspace:
28       host: https://e2-demo-west.cloud.databricks.com
29     resources:
30       jobs:
31         shark_sightings:
32           schedule:
33             quartz_cron_expression: 14 8 14 * * ?
34             timezone_id: UTC
```



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

A closer look at the bundle.yml file reveals its structured nature.

It starts by defining the name and default Workspace, establishing the context for the project.

Next, it specifies the Resource configurations. These define resources such as Jobs, DLT pipelines, or MLflow components. This resource section follows the REST API schema for configuration.

Finally, and most critically for LLM Ops, it contains Environment-based specs. These specifications allow you to control project behavior—such as cluster size, schedule, or access permissions—differently across environments like staging and production.

Where Can We Use Bundles?

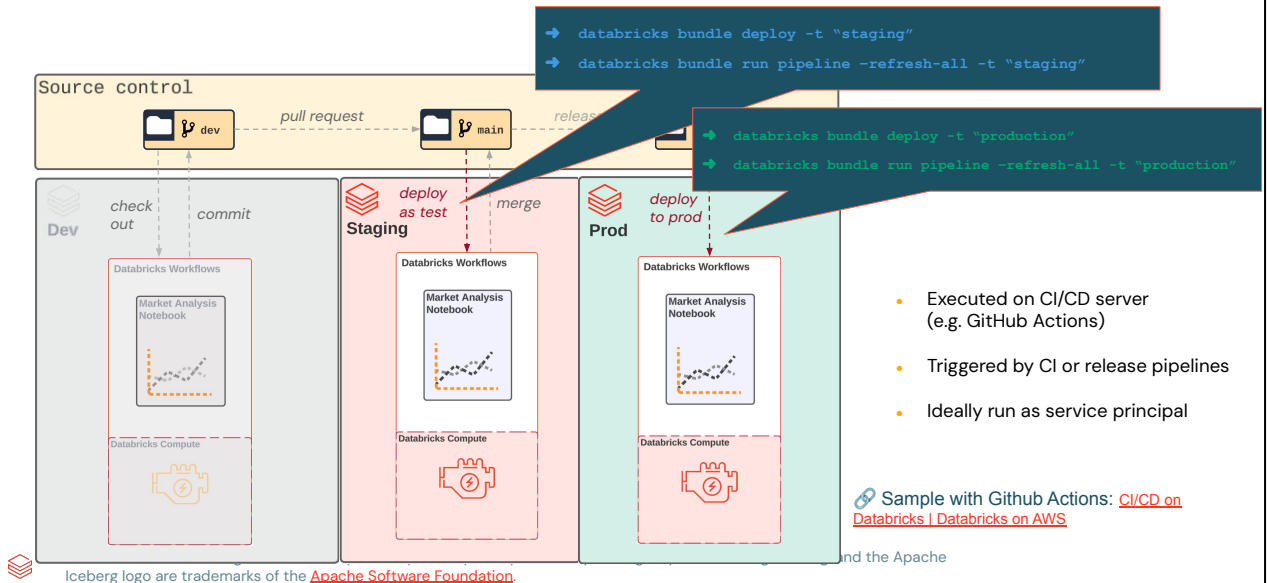
As part of active development



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

1. Bundles are integral to the active development phase.
2. As a developer, you can use simple CLI commands, such as `databricks bundle deploy -t "development"`, to deploy your project, tweak configurations, and immediately test changes.
3. You can deploy and run your project from various interfaces like IDEs, terminals, or Databricks notebooks. This functionality allows for deploying to multiple workspaces for testing specific differences or isolation.

Where can we use bundles?



When moving beyond development, bundles become the core of your automation pipeline.

Bundles are executed on a CI/CD server, such as one running GitHub Actions. They are triggered automatically by CI or release pipelines whenever code is merged or promoted.

The commands, like `databricks bundle deploy -t "staging"` or `databricks bundle deploy -t "production"`, target the specific configurations for those environments, as defined in the `bundle.yml` file.

It is highly recommended that these automated runs be executed as a service principal rather than an individual user to ensure secure, reliable, and traceable deployments.

Benefits of Using DABs

Write code once, deploy anywhere

- **Democratizing best practices for CI/CD and project management**
 - Co-version configs with code and resources
 - Automation friendly (CLI vs. 1000 REST calls)
 - Collaborate and deploy with confidence (isolation, automated testing, etc)
- **Unified, single way to define and deploy your projects**



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

The overarching benefit of using Databricks Asset Bundles is the unified approach to project deployment.

DABs are essential for democratizing best practices for CI/CD and project management. They achieve this by enabling you to co-version configurations alongside code and resources.

This approach is highly automation friendly, translating potentially thousands of individual REST API calls into simple CLI commands.

Ultimately, DABs provide a unified, single way to define and deploy your projects, allowing teams to collaborate and deploy with confidence through isolation and automated testing.



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

Thank you for completing this lesson and continuing your journey to develop your skills with us.